UTN FRLP

Desarrollo de Software 2026

Persistencia de Datos

Clase #16 — 10 de agosto de 2026

Matías Corti - Martina García Améndola



Dónde quedamos

La clase pasada dejamos andando el CRUD de `/api/libros` y `/api/autores` en cuatro capas, con el `api.http` commiteado.

Desde C13 tenemos prendido el servicio `db`, con su volumen `pgdata`, sin pedirle nada.

Pendientes de C15	**Lo resuelve**
Reiniciás y los libros se van	← HOY
POST sin título guarda basura	C17 · Zod
/libros/abc da 404 en vez de 400	C17 · validación
libro.autor es texto, no un Autor	C17 · relaciones
Cualquiera borra cualquier libro	C18 · Auth con JWT
Tu front sigue leyendo libros.json	C19 · front ↔ back

Temas de hoy

- **Por qué una base de datos:** RAM vs disco, concurrencia, SQL vs NoSQL
- **Dónde vive el dato en Docker:** el volumen `pgdata`, `down` vs `down -v`
- **ORM:** qué resuelve, qué no resuelve, y por qué usamos Prisma
- **Schema, migraciones y cliente generado:** las tres piezas de Prisma
- **`async`/`await` en el backend:** el cambio que se propaga por todas las capas
- **Práctica:** reemplazar el array en memoria por PostgreSQL, sin tocar el contrato de la API



¿QUÉ PASA CUANDO...?

- `docker compose restart api`
- Se te cierra la terminal
- Cambiás una línea y `tsx watch` recarga
- Se corta la luz

💭 **...tus datos DESAPARECEN.**

El array vive en la RAM del proceso Node.

El proceso muere, el array muere.



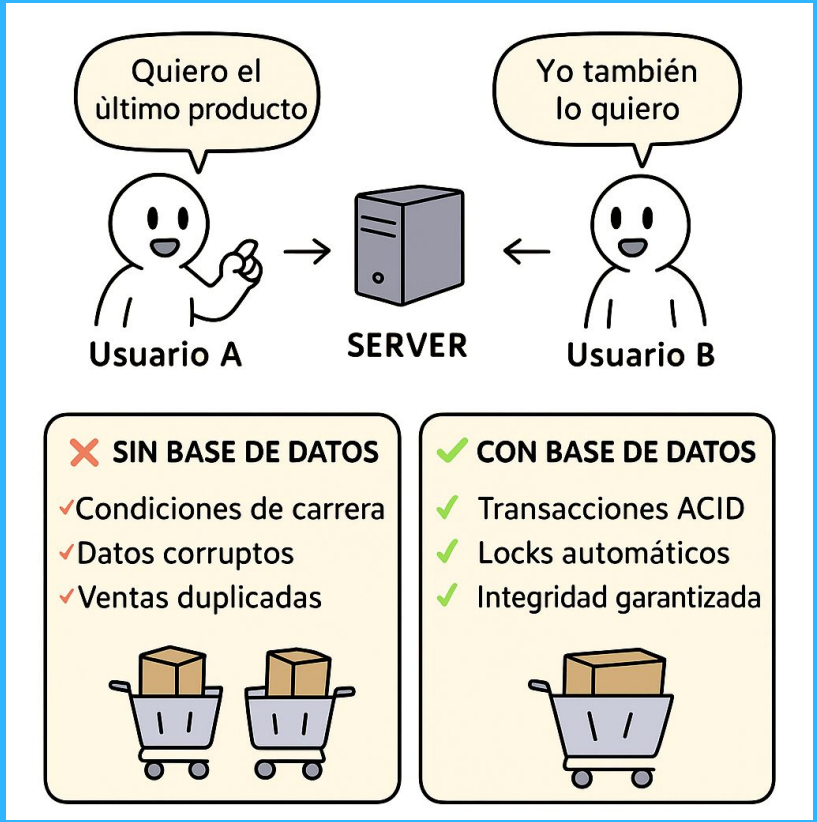
RAM vs Disco

¿Qué es? Dos lugares donde puede vivir un dato, con propiedades opuestas.

Una base de datos usa las dos: disco para que el dato no se pierda, RAM para que la consulta sea rápida. Y además resuelve lo que un archivo no: **concurrencia** (diez personas comprando el último ejemplar a la vez), **consultas** ("los disponibles de menos de $6000, por precio") e **integridad**.

	🧠 **Memoria RAM**	💾 **Disco**
Velocidad	✅ Nanosegundos	❌ Milisegundos
Al apagarse	❌ Se pierde (volátil)	✅ Queda (persistente)
Capacidad	❌ Limitada	✅ Alta
Costo por GB	❌ Caro	✅ Barato (~50x)

🔄 Concurrencia



*Las bases de datos **no solo guardan datos: garantizan que sean correctos** cuando todos los usuarios **compiten al mismo tiempo***

⚔️ SQL vs NoSQL

🏛️ SQL (Relacional)	🌐 NoSQL (Documental)
Tablas con esquema fijo	Documentos flexibles
Relaciones fuertes (JOINs)	Escalamiento horizontal fácil
Consultas complejas	Desarrollo rápido, sin migrar
Transacciones ACID	Ideal para datos que cambian de forma
PostgreSQL, MySQL, SQL Server	MongoDB, Firebase, DynamoDB

📊 Tablas Estructuradas

ID	Name	Age
1	Juan	25
2	Ana	30

📄 Documentos Flexibles

```
{
  "id": 1,
  "name": "Juan",
  "age": 25,
  "hobbies": ["code", "games"]
}
```

Para este curso utilizaremos SQL (Postgres)
Pero es importante conocer las NoSQL



¿Tabla o documento?

Escenarios. Votan a mano alzada: SQL o NoSQL.

⏱️ **5 minutos**

1. El catálogo de libros de su librería

2. Los logs de todo lo que hace la app (millones por día, cada uno con campos distintos)

3. Las facturas de venta, que tienen que cuadrar con los pagos

4. El chat de soporte: mensajes con texto, imágenes, audios, encuestas

5. Usuarios, roles y permisos

6. El carrito de compras de una sesión que todavía no se confirmó



Dónde vive el dato (en Docker)

El dato de PostgreSQL vive en un **volumen**, no en el container.

El container es descartable. El volumen es lo que no.

```yaml
db:
  image: postgres:16-alpine
  volumes:
    - pgdata:/var/lib/postgresql/data       # ← acá vive TODO
volumes:
  pgdata:                # ← volumen nombrado, lo gestiona Docker
```

Comando	Qué pasa con tus datos
`docker compose restart db`	Siguen ahí
`docker compose down → up`	Siguen ahí: el volumen no se toca
`docker compose down -v`	Se borran: la -v borra el volumen
Tocás src/ y recarga tsx	Siguen ahí

🔁

Mini-cierre: por qué una DB

Concepto	En una línea
RAM	Rápida y volátil: muere con el proceso
Disco	Lenta y persistente: sobrevive al apagón
Base de datos	Usa las dos, y resuelve concurrencia, consultas e integridad
Concurrencia	Muchos escribiendo a la vez sin corromper el dato
SQL	Forma estable + relaciones → PostgreSQL
NoSQL	Forma variable + escala horizontal → MongoDB
Volumen (pgdata)	Donde vive el dato: sobrevive al container
down vs down -v	Recrear el container vs BORRAR la base



ORM



¿Que es un ORM?

Object-Relational Mapping

Traductor entre el mundo de los Objetos y el Relacional

*Es una herramienta que actúa como un **puente** entre: la **programación orientada a objetos** (POO) y las **bases de datos relacionales**.*

Su principal objetivo es permitirte interactuar con la base de datos utilizando objetos y clases de tu lenguaje de programación, en lugar de escribir consultas SQL directamente.

Principales responsabilidades:

- **Mapeo:** Tablas ↔ Clases; Columnas ↔ Atributos.
- **Hidratado:** Convertir registros de la BD en objetos.
- **Persistencia:** Guardar objetos en la BD.
- **Conversión y validación** de datos antes de consultas/updates.
- **Abstracción:** Independencia (parcial) del motor de BD.



¿QUÉ ES UN ORM?

¿Por qué?

Sin ORM, cada consulta es un string de SQL: sin autocompletado, sin chequeo de tipos, con los errores apareciendo en tiempo de ejecución.

En TypeScript	En PostgreSQL
Clase / interface Libro	Tabla Libro
Propiedad titulo	Columna titulo
Objeto { id: 1, titulo: "1984" }	Fila
libros: Libro[]	Resultado de un SELECT



SQL crudo vs Prisma

La diferencia no es de estilo: sin ORM, la consulta es un **string** y TypeScript no sabe nada de ella.

```sql
-- Sin ORM (SQL crudo)
SELECT id, titulo, precio FROM "Libro"
WHERE disponible = true AND precio < 6000
ORDER BY precio ASC;
-- en tu código:
-- const filas = await db.query('SELECT id, titulo...', [true]);
-- filas es any[]. TypeScript no sabe nada.
```

```ts
// Con ORM (Prisma)
const libros = await prisma.libro.findMany({
  where: { disponible: true, precio: { lt: 6000 } },
  orderBy: { precio: "asc" },
});
// libros es Libro[]. Autocompletado, chequeo de tipos, y si te
// equivocás en un nombre de campo no compila.
```



QUÉ NO RESUELVE UN ORM

Un ORM no es gratis. Lo que **no** te da:

- **No te ahorra entender SQL.** Genera SQL. Si no sabés qué genera, no sabés por qué anda lento
- **No te hace independiente del motor.** Cambiar de PostgreSQL a MySQL no es sólo cambiar una línea
- **No optimiza tus consultas.** Pedir mal es pedir mal, con o sin ORM
- **No diseña tu modelo de datos.** Eso es trabajo tuyo, y ya lo hicieron en C13
- **Agrega una capa más:** una dependencia, una versión, un lugar más donde puede romperse

Vale la pena igual. Pero elegir una herramienta sabiendo qué te cuesta es distinto a elegirla porque es la que aparece en el tutorial.



Las tres piezas de Prisma

Prisma son tres cosas distintas y conviene no confundirlas. **El `schema.prisma` es la única fuente de verdad:** todo lo demás se deriva de él.

La **migración se commitea**; el **cliente generado no**, porque es código derivado (como `node_modules`).

Pieza	Qué es	Quién
schema.prisma	La definición de tus tablas	VOS
Migración	El SQL que lleva la DB de A a B	Prisma
Cliente generado	El código TS y sus tipos	Prisma

```
schema.prisma
    │
    ├──> prisma migrate dev ──> SQL ──> tu base de datos
    │
    └──> prisma generate ───> src/generated/prisma/ (tipos + métodos)
```



Nuestro modelo: schema.prisma

Cada `model` es una tabla, y el nombre va en singular.

- `@id` = clave primaria
- `@default(autoincrement())` = el id lo pone la base
- `@default(true)` = si no viene, vale true
- `@unique` = no se puede repetir.

```
// backend/prisma/schema.prisma

model Libro {
  id         Int     @id @default(autoincrement())
  titulo     String
  autor      String        // por ahora, texto. La relación en C17
  precio     Int
  imagen     String
  disponible Boolean @default(true)
}

model Autor {
  id           Int    @id @default(autoincrement())
  nombre       String @unique
  nacionalidad String
}
```

⚠️

Prisma 7: dónde va la URL

En Prisma 6 (y en todos los tutoriales) la conexión iba en el schema. **En Prisma 7 va en un archivo nuevo.**

En internet van a ver información de versiones viejas. El 99% de lo que leyó es de Prisma 5 y 6.

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")  // ❌ ESTO YA NO EXISTE en Prisma 7
}
// error: The datasource property `url` is no longer supported in
schema files.
```

```
// backend/prisma.config.ts
import { defineConfig } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: {
    path: "prisma/migrations",
    seed: "npx tsx prisma/seed.ts",
  },
  datasource: {
    url: process.env.DATABASE_URL,    // Compose la inyecta desde .env
  },
});
```

🔗 Anatomía del connection string

db no existe en tu máquina. Es un nombre que sólo resuelve dentro de la red que armó Docker Compose: adentro del container `api` el host es **db**; desde tu máquina, `localhost:5432`.

🔴 **Por eso todos los comandos de Prisma se corren adentro del container:**

```
docker compose exec api npx prisma ...
```

```
postgresql://postgres:tu_password_segura@db:5432/libreria_db
└────────┘   └──────┘ └──────────────────┘ └┘ └┘ └──────────┘
  motor       usuario      password      host puerto   base

# backend/.env  —  NO va a Git   (armado por partes, desde C13)
POSTGRES_USER=postgres
POSTGRES_PASSWORD=tu_password_segura
POSTGRES_DB=libreria_db
POSTGRES_PORT=5432
DATABASE_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:${PO
STGRES_PORT}/${POSTGRES_DB}
```

🔄 Migraciones

¿Qué es? El control de versiones de la **estructura** de tu base de datos. Git para las tablas.

Qué hace, en orden:

1) compara el `schema.prisma` con la base
2) genera el SQL que falta
3) lo aplica
4) lo guarda con fecha y nombre

Cambiás el schema → `migrate dev --name agrega_stock` → una migración nueva. **El historial no se reescribe: se le suma.**

```
# 1) Crear y aplicar la migración
docker compose exec api npx prisma migrate dev --name init
```

```
# 2) Generar el cliente TypeScript ← comando APARTE
docker compose exec api npx prisma generate
```

```
backend/prisma/
├── schema.prisma
└── migrations/
    ├── 20260810143022 init/
    │   └── migration.sql          ← el SQL, commiteado
    └── migration_lock.toml
```



Los tipos ahora los genera Prisma

Prisma genera los tipos de TypeScript **a partir del schema.** Ya no los declarás a mano.

La capa `types/` no desaparece: cambia de rol. Antes *declaraba* la forma; ahora *le pone nombre* a la forma que declara el schema.

✅ **El resto del backend no cambia ni un import:** `services` y `controllers` siguen escribiendo `import { Libro } from "../types/libro.types"`.

```
// ANTES - backend/src/types/libro.types.ts (C15)
export interface Libro {
  id: number; titulo: string; autor: string;
  precio: number; imagen: string; disponible: boolean;
}
```

```
// AHORA - backend/src/types/libro.types.ts (C16)
// El schema es la única fuente de verdad.
// Acá sólo le ponemos el nombre corto.
export type { LibroModel as Libro } from "../generated/prisma/models";
```



ASYNC/AWAIT: EL CAMBIO QUE SE PROPAGA

Prisma devuelve una Promise: la consulta viaja por la red y eso tarda. Es el mismo async/await de front y de los useEffect, ahora del lado backend. **Y se contagia hacia arriba:** si el service es async, el controller tiene que await.

El bug de hoy: te olvidás un await, la respuesta es {} y **no hay ningún error**. Ni de tipos, ni en runtime. Si tu endpoint devuelve {}, buscá el await que falta.

```
// C15: el array estaba en memoria. Instantáneo.
export function findAll(): Libro[] {
  return libros;
}

// C16: hay que ir hasta PostgreSQL y volver.
export async function findAll(): Promise<Libro[]> {
  return prisma.libro.findMany();
}
```

```
// ❌ Sin await
const libro = libroService.findById(3);
if (!libro) return res.status(404)...  // una Promise NO es inmediata
return res.json(libro);                // manda {}  ← ni error, ni 404
```



El cliente: una sola instancia

¿Por qué una sola? Cada `new PrismaClient()` abre su propio pool: uno por request y la base se queda sin conexiones. 📁 **Nace la carpeta `config/`.**

⚠️ **Novedad de Prisma 7:** hay que pasarle un **adapter**. Antes traía su propio motor; ahora usa el driver `pg` de Node: el `@prisma/adapter-pg` que instalamos.

```
// backend/src/config/prisma.ts
import { PrismaClient } from "../generated/prisma/client";
import { PrismaPg } from "@prisma/adapter-pg";

// El adapter es el driver: el que realmente habla TCP con PostgreSQL.
const adapter = new PrismaPg({ connectionString:
process.env.DATABASE_URL });

// Una sola instancia para toda la app: abre un pool de conexiones y lo
reusa.
export const prisma = new PrismaClient({ adapter });
```



Prisma Studio

Un cliente visual para tu base, sin instalar nada. Abrís http://localhost:5555 en tu navegador: ver los datos, editarlos como una planilla, crear y borrar filas, filtrar y buscar.

Para que funcione hace falta mapear el puerto en el compose.

Alternativa si preferís SQL crudo: docker compose exec db psql -U postgres -d libreria_db

```
docker compose exec api npx prisma studio --port 5555 --browser none
```

```
api:
  ports:
    - "3000:3000"
    - "5555:5555"   # ← Prisma Studio
```

🔁

Mini-cierre: ORM y Prisma

Concepto	En una línea
ORM	Traductor objetos ↔ tablas relacionales
schema.prisma	Fuente de verdad: model = tabla
prisma.config.ts	Dónde está el schema y la URL (Prisma 7)
Migración	Git de la estructura. Se commitea
Cliente generado	src/generated/prisma/. No va a Git
config/prisma.ts	Una instancia con adapter para la app
@db vs localhost	En el container vs desde tu máquina
async/await	Prisma da Promises: service → controller



PRÁCTICA



Antes de empezar

INSTALAR PRISMA

```bash
# 1) Agregar las dependencias al proyecto
cd backend
npm install prisma@7.9.1 --save-dev
npm install @prisma/client@7.9.1
npm install @prisma/adapter-pg pg

# 2) Reconstruir la imagen  ← este paso NO es opcional
cd ..
docker compose build api

# 3) Verificar que quedó adentro del contenedor
docker compose up -d
docker compose exec api npx prisma --version
```



Paso 0: abrirle la puerta a Prisma

10 minutos. El compose sólo monta `src/`, y Prisma va a escribir en `prisma/` → **hay que mapearla**. Sumá también `CHECKPOINT_DISABLE=1` y `PRISMA_HIDE_UPDATE_MESSAGE=1` al final de `.env` **y** de `.env.example` (sin telemetría: no cuelga sin internet).

Antes de levantar: `mkdir -p backend/prisma && touch backend/prisma.config.ts` — si no existen, Docker crea **carpetas** y se rompe. Después, `docker compose up -d`. **Listo cuando** `docker compose ps` muestra `api` y `db` arriba, y `db` dice `healthy`.

```yaml
# docker-compose.yml  (raíz del proyecto) — lo que se agrega
services:
  api:
    ports: ["3000:3000", "5555:5555"]                    # ← 5555 = Studio
    volumes:
      - ./backend/prisma:/app/prisma                     # ← NUEVO
      - ./backend/prisma.config.ts:/app/prisma.config.ts  # ← NUEVO
    depends_on:
      db: { condition: service_healthy }                 # ← que esté LISTA
  db:
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $$POSTGRES_USER -d $$POSTGRES_DB"]
      interval: 5s
      retries: 10
```

Ejemplo Completo: DS26-Libreria (C16)



Paso 1: prisma.config.ts + schema

12 minutos. Dos archivos, se escriben una vez y no se vuelven a tocar. El `prisma.config.ts` es exactamente el de la filmina 18, y los dos `model` son los de la filmina 17.

Y una línea a `backend/.gitignore`: `src/generated/prisma`. El cliente generado es código derivado, si lo commitean, van a tener conflictos de merge sobre archivos que nadie escribió.

Listo cuando: `docker compose exec api npx prisma validate` responde *The schema at prisma/schema.prisma is valid* 🚀

```
// backend/prisma/schema.prisma
generator client {
  provider = "prisma-client"      // ← Prisma 7. NO "prisma-client-js"
  output   = "../src/generated/prisma"
}

datasource db {
  provider = "postgresql"
  // sin url: en Prisma 7 va en prisma.config.ts (filmina 17)
}

// + los dos model de la filmina 16: Libro y Autor -> actualizar types
```

Ejemplo Completo: DS26-Libreria (C16)



Paso 2: la primera migración

10 minutos. Corré el comando, y después **mirá qué pasó** en tu proyecto, adentro de la migración, y en la base.

Listo cuando: existe la carpeta migrations/, y la base lista las tres tablas.

```
# 1) Crear y aplicar la migración
docker compose exec api npx prisma migrate dev --name init
```

```
# 2) Generar el cliente TypeScript ← comando APARTE
docker compose exec api npx prisma generate
```

```
1) en tu proyecto
   backend/prisma/migrations/20260810143022_init/migration.sql

2) adentro de la migración
   CREATE TABLE "Libro" (
       "id" SERIAL NOT NULL,
       "titulo" TEXT NOT NULL, ...
       CONSTRAINT "Libro_pkey" PRIMARY KEY ("id")
   );

3) en la base
   docker compose exec db psql -U postgres -d libreria_db -c "\dt"
   public | Autor | table    public | Libro | table
   public | _prisma_migrations | table
```

Ejemplo Completo: DS26-Libreria (C16)



Paso 3: cliente + seed

12 min. El cliente es config/prisma.ts
El seed: docker compose exec api npx prisma db seed.

Sin id (si no, el contador queda en 1 y el primer POST explota).
Una sola vez (dos = 20 libros; para volver: docker compose exec api npx prisma migrate reset -f).

```
// backend/prisma/seed.ts
import { prisma } from "../src/config/prisma";
const libros = [
  { titulo: "El principito", autor: "Antoine de Saint-Exupéry",
    precio: 4500, imagen: "https://...", disponible: true },
  // ... los 10 de C15, COPIADOS DEL SERVICE, y SIN id
];
const autores = [
  { nombre: "Antoine de Saint-Exupéry", nacionalidad: "Francia" },
  // ... Gabriel García Márquez y Ernesto Sabato
];
async function main() {
  await prisma.libro.createMany({ data: libros });
  await prisma.autor.createMany({ data: autores });
}
main();
```

Ejemplo Completo: DS26-Libreria (C16)



Paso 4: el service, a Prisma

15 min. Mismos nombres, mismas firmas, mismos retornos, sólo cambia de dónde salen los datos.

- `filter(…)` → `findMany({ where })`
- `find(…)` → `findUnique({ where: { id } })`
- `push(…)` → `create({ data })`
- `Libro | undefined` pasa a ser `Libro | null`.

```ts
// backend/src/services/libro.services.ts
// el array de 10 libros y el proximoId: SE VAN. Ahora viven en la base.
export async function findAll(disponible?: boolean): Promise<Libro[]> {
  return prisma.libro.findMany({ where: { disponible } });
}
export async function findById(id: number): Promise<Libro | null> {
  return prisma.libro.findUnique({ where: { id } });
}
export async function create(datos: Omit<Libro, "id">): Promise<Libro> {
  return prisma.libro.create({ data: datos });
}
export async function remove(id: number): Promise<boolean> {
  const existe = await prisma.libro.findUnique({ where: { id } });
  if (!existe) return false;                    // update: igual — buscar,
  await prisma.libro.delete({ where: { id } });  // y después update()
  return true;
}
```

Ejemplo Completo: DS26-Libreria (C16)



Paso 5: el controller, con await

8 min. Mismos status codes que C15: se le suma async, await y un try/catch. **Y así los cinco.** Sí: el mismo try/catch cinco veces.

😤 **Ese copy-paste es una deuda:** cinco catch idénticos por entidad, diez con autores. En C17 se van los diez y queda **un** middleware de error.

```
// backend/src/controllers/libro.controller.ts
export async function getById(req: Request, res: Response) {
  try {
    const libro = await libroService.findById(Number(req.params.id));
    if (!libro) return res.status(404).json({ error: "Libro no encontrado"
});
    return res.json(libro);
  } catch (error) {
    return res.status(500).json({ error: "Error interno del servidor" });
  }
}
export async function create(req: Request, res: Response) {
  try {
    const nuevo = await libroService.create(req.body);
    return res.status(201).json(nuevo);
  } catch (error) { /* ídem: 500 + mensaje genérico */ }
}
```

Ejemplo Completo: DS26-Libreria (C16)



Paso 6: el api.http de C15, tal cual

5 min. Abrí tu `api.http` de la semana pasada, **no le cambies nada** y corrélo entero. Después `docker compose restart api` y volvé a pedirlo: **el libro que creaste sigue ahí.**

🎯 **La API no cambió: ni una ruta, ni un verbo, ni un status.** Cambió lo que hay atrás. **Eso es para qué separamos en capas.**

Request	Status esperado
GET /api/libros	200 · los 10 libros
GET /api/libros?disponible=true	200 · sólo los disponibles
GET /api/libros/1 · /999	200 · 404
POST /api/libros	201 ← ojo acá
PUT /1 · DELETE /3	200 · 204

Ejemplo Completo: DS26-Libreria (C16)

🔁

Mini-cierre: la práctica

Lo que hicimos	En una línea
Volúmenes nuevos	prisma/ y prisma.config.ts, mapeados
healthcheck	El api espera a que la DB esté LISTA
schema.prisma	Dos modelos: Libro y Autor
migrate dev --name init	Creó las tablas y guardó el SQL en Git
db seed	Los 10 libros en la base, sin id a mano
config/prisma.ts	Una instancia con adapter para la app
El service	De array.find() a Prisma, y async
El controller	await + try/catch, mismos status
El api.http	NO se tocó. Los datos ya sobreviven



Lo que falta, falta a propósito

Hoy nos llevamos: una API con datos que sobreviven, migraciones versionadas y tipos generados desde una única fuente de verdad.

Pendiente de hoy	Lo resuelve
Libro.autor es String, sin relación	C17 · @relation
POST sin titulo guarda basura	C17 · Zod
/libros/abc da 404 en vez de 400	C17 · validación
Diez try/catch, y todo error es 500	C17 · middleware
Dos autores iguales dan 500, no 409	C17 · errores Prisma
Cualquiera borra cualquier libro	C18 · Auth con JWT
Tu front sigue leyendo libros.json	C19 · front ↔ back

Actividades

Actividad Individual

1° instancia hasta el **23/08/2026**
*2° instancia: **13/09/2026** (Cierre Bloque 3)*

[AI-C16] Persistencia en Librería

Referencia en el repo de ejemplo branch C16

1. En ds2026-actividades: copiar el sitio de la librería de la Clase 15 en **ejercicios/c16-persistencia/**

2. docker-compose.yml: los volúmenes de prisma/ y prisma.config.ts, y el healthcheck de db

3. schema.prisma con **Libro** y **Autor**, sin relación entre ellos

4. **Al menos una migración** con nombre descriptivo, commiteada

5. prisma/seed.ts con tus datos (sin id a mano)

6. **Las dos entidades** conectadas a la base: service y controller de Libro y de Autor

7. api.http corriendo entero, casos de error incluidos (los 404)

8. .env.example actualizado (con placeholders) y .env **fuera de Git**

9. Commiteá y avisá en #actividades
   → hilo **[AI-C16] Persistencia en Librería**

Actividad grupal

Hasta el **23/08/2026**
(inclusive)

[AG-C16] Persistencia en el TP

1. Traducir el modelo de datos del TP schema.prisma.

2. Mínimo 3 entidades con migrations. Sin relaciones todavía (eso en C17)

3. service y controller de **al menos una entidad** conectados a la base (de las implementadas en C15)

4. prisma/seed.ts con datos de prueba del dominio de su cliente

5. .env.example actualizado. .env fuera de Git. **Cero credenciales reales en el repo**

4. Dividir las tareas en el equipo, **hacer PRs con reviews cruzados** (al menos uno cada alumno) y en trello mover tarjetas correspondientes a "Done"

5. Aviso en #actividades
   → hilo **[AG-C16] Persistencia en el TP**

⚠️ Antes de la próxima clase

⚠️ INSTALAR ANTES DE C17

El aula no tiene internet confiable.

Sin instalación = sin práctica.

```bash
# 1) Agregar las dependencias al proyecto
cd backend
npm install zod@4.4.3

# 2) Reconstruir la imagen  ← este paso NO es opcional
cd ..
docker compose build api

# 3) Verificar que quedó adentro del contenedor
docker compose up -d
docker compose exec api npm ls zod
```

✔ Dudas → canal #consultas de Discord

📅 C17: Relaciones y Validaciones (24/08/2026)